

Lenguajes para Análisis de Datos II

Modulo 2

Objetivo

En esta unidad se abordan una variedad de conceptos y técnicas clave en el análisis de datos utilizando la biblioteca Pandas en Python. Desde la manipulación de DataFrames hasta la realización de operaciones avanzadas como la fusión y agregación de conjuntos de datos, el contenido abarca un amplio espectro de habilidades esenciales para cualquier profesional que trabaje con datos.



¿Qué es Pandas?

En el mundo de la ciencia de datos, existen una gran variedad de librerías de Python para elegir, pero hay una que destaca sobre el resto en el ámbito del análisis de datos y la manipulación de estructuras de datos: Pandas.

Pandas fue creada para facilitar y mejorar el manejo de datos estructurados, especialmente en forma de tablas y series de tiempo. Proporciona dos estructuras de datos fundamentales que permiten representar y trabajar con datos de manera eficiente: **Dataframe** y **Series**.



Usos de Pandas

Pandas es ampliamente utilizada en el análisis de datos y la ciencia de datos debido a su versatilidad y funcionalidad. Algunas de las aplicaciones para las que se utiliza son:

- **Carga y manipulación de datos:** Pandas permite cargar datos desde diferentes fuentes, como archivos CSV, Excel, bases de datos y más. También proporciona herramientas para limpiar y preprocesar datos, como el manejo de datos faltantes y duplicados.
- **Análisis exploratorio de datos:** Pandas facilita el análisis y la exploración inicial de los datos, permitiendo realizar consultas, filtrar, ordenar y agregar datos de manera sencilla.
- **Operaciones de datos:** Pandas proporciona una amplia gama de funciones y métodos para realizar operaciones matemáticas, estadísticas y de manipulación de datos en las estructuras DataFrame y Series.
- **Visualización de datos:** Pandas se integra perfectamente con librerías de visualización como Matplotlib y Seaborn, lo que permite crear gráficos y visualizaciones de datos de manera sencilla.
- **Integración con otras herramientas de ciencia de datos:** Pandas se utiliza comúnmente en conjunto con otras librerías de Python, como NumPy, Scikit-learn y TensorFlow, para realizar análisis y modelado de datos más complejos.



Operar con Dataframes

Saber cómo cargar y guardar DataFrames en Python es una habilidad fundamental para cualquier científico de datos o analista que trabaje con esta herramienta. Estas operaciones son esenciales para acceder a los datos, compartir información, optimizar tu flujo de trabajo y realizar análisis de datos de manera efectiva.

The diagram illustrates a DataFrame structure. A table with 7 rows and 6 columns is shown. The columns are labeled 'Name', 'Team', 'Number', 'Position', and 'Age'. The rows are indexed from 0 to 6. Annotations include: 'Columns' with arrows pointing to the column headers; 'Rows' with arrows pointing to the row indices; and 'Data' with a bracket pointing to the data cells. Some cells are highlighted with pink boxes: 'Jonas Jerebko', '8.0', 'Boston Celtics' (in row 2), 'NaN' (in row 3), 'PG' (in row 4), and 'NaN' (in row 5).

	<i>Name</i>	<i>Team</i>	<i>Number</i>	<i>Position</i>	<i>Age</i>
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Cargar Dataframes

Para poder trabajar con DataFrames en Python, lo primero que hay que hacer es cargar el archivo que contiene los datos. Pandas ofrece funciones específicas para cargar datos en diferentes formatos, siendo los más comunes CSV y Excel.

- **Cargando un archivo .csv**

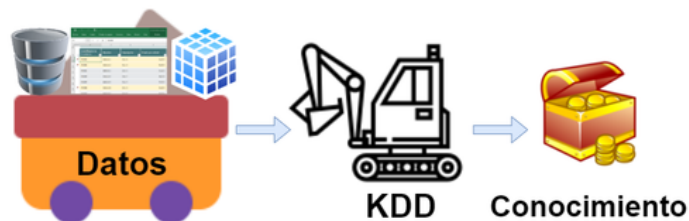
Import pandas as pd

```
DF = pd.read_csv("Mi_Archivo.csv")
```

- **Cargando un archivo .xlsx**

Import pandas as pd

```
DF = pd.read_excel("Mi_Archivo.xlsx")
```



Guardar Dataframes

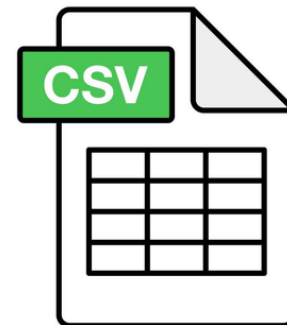
Una vez que has realizado tu análisis y procesado los datos en un DataFrame, es importante guardarlos para poder consultarlos o utilizarlos en el futuro. Pandas ofrece funciones específicas para guardar DataFrames en diferentes formatos, siendo los más comunes CSV y Excel.

- **Guardando un archivo en formato .csv**

```
DF.to_csv("Mi_Archivo.csv")
```

- **Guardando un archivo en formato .xlsx**

```
DF.to_excel("Mi_Archivo.xlsx")
```



Analizar datos con Pandas

Hoy en día, es posible extraer información muy valiosa de las cantidades masivas de datos que nos rodean. Sin embargo, esta información rara vez se obtiene de un simple vistazo. Por esta razón, necesitamos herramientas como Pandas para realizar análisis complejos y crear visualizaciones efectivas. Esta biblioteca destaca por su interfaz intuitiva y su amplia gama de funcionalidades, lo que la convierte en una herramienta esencial para cualquier científico de datos.



Acceder a los nombres de las columnas

Los nombres de las columnas de un DataFrame se pueden obtener mediante `Data.columns`. Esta función devuelve un objeto `Index` que contiene los nombres de todas las columnas del DataFrame.

`Data.columns`

Acceder a columnas específicas

Para acceder a los datos de una columna específica, se utiliza la notación de corchetes `[]`. Para hacerlo, simplemente coloca el nombre de la columna dentro de los corchetes.

`Data["nombre_columna"]`

`Data["Element"]`

	total_bill	tip	smoker	day	time	size
0	16.99	1.01	No	Sun	Dinner	2
1	10.34	1.66	No	Sun	Dinner	3
2	21.01	3.50	No	Sun	Dinner	3
3	23.68	3.31	No	Sun	Dinner	2
4	24.59	3.61	No	Sun	Dinner	4

Conocer los tipos de datos

La función `Data.dtypes` devuelve una Serie que muestra el tipo de dato de cada columna en el `DataFrame`, lo que es útil para entender cómo están almacenados los datos.

```
Data.dtypes
```

Crear columnas con valores únicos

Puedes crear una nueva columna y asignar un valor único a todas las filas del `DataFrame`. Para hacerlo, utiliza la notación de corchetes para especificar el nombre de la nueva columna y asigna el vector. Esto es útil para agregar información o realizar operaciones lógicas.

```
Data["logica"] = False
```

Conocer las dimensiones del dataset

La función `Data.shape` devuelve una tupla que contiene el número de filas y el número de columnas del `DataFrame`. Es útil para obtener una idea rápida del tamaño del dataset.

```
Data.shape
```



Explorar las primeras filas

Con `Data.head()`, puedes ver las primeras n filas del `DataFrame`. Por defecto, muestra las primeras 5 filas, pero puedes especificar otro número de filas.

```
Data.head(número_de_filas)
```

```
Data.head(7)
```

Explorar las últimas filas

Similar a `head()`, la función `Data.tail()` permite ver las últimas n filas del `DataFrame`.

```
Data.tail(número_de_filas)
```

```
Data.tail(7)
```



Obtener un resumen del dataset

La función `Data.describe()` proporciona un resumen estadístico de las columnas numéricas del `DataFrame`, incluyendo la media, la desviación estándar, el mínimo, el máximo y los percentiles.

```
Data.describe()
```

Filtrado y manipulación de datos con Pandas

Objetivo

Comprender que en el análisis de datos, filtrar y manipular datos son técnicas esenciales para extraer información útil de grandes conjuntos de datos. Usando Pandas, puedes realizar estas operaciones de manera eficiente, lo que te permite centrarte en los datos más relevantes y prepararlos para un análisis más detallado.



- El **filtrado** permite seleccionar subconjuntos específicos dentro de todo el contenido del dataset. En el ejemplo del Dataframe de la tabla periódica, es posible que sólo queramos emplear datos de elementos radioactivos, elementos descubiertos en una época concreta o elementos que tengan un cierto número atómico. Esta selección acelera y simplifica el proceso para seleccionar únicamente los datos que necesitamos para seguir con el análisis.
- El **manipulado** de datos no se refiere a cambiarlos para que refuercen ideas que tengamos, sino a cambiar el tipo de los datos para poder realizar ciertas operaciones, eliminar elementos duplicados, reorganizar en un orden específico o renombrar columnas para facilitar la comprensión. En el ejemplo del Dataframe de la tabla periódica, esto se podría traducir en eliminar una columna que contiene la misma información que otra u organizar los elementos según su masa atómica o fecha de descubrimiento.



Beneficios

A continuación, se presenta un análisis detallado de los beneficios que ofrece la combinación del filtrado y la manipulación de datos en Pandas:

- **Extraer los datos relevantes para el análisis:** El filtrado te permite aislar los datos específicos que son pertinentes para tu análisis, descartando aquellos que no aportan valor o que podrían generar distorsiones. Esto es especialmente útil cuando trabajas con grandes volúmenes de datos.
- **Preparar los datos para el análisis:** La manipulación de datos te permite transformar, limpiar y organizar los datos para que sean aptos para el análisis.
- **Obtener información precisa:** Combinando el filtrado con la manipulación, puedes asegurarte de que los datos sean precisos y relevantes, lo que resulta en conclusiones más confiables.
- **Facilitar la comprensión:** Estas operaciones pueden simplificar la estructura del dataset, haciéndolo más fácil de interpretar y permitiendo una identificación más rápida de patrones y tendencias.



Obtención de columnas o filas

En Pandas, la obtención de columnas o filas específicas de un dataset es una operación fundamental, especialmente cuando se trabaja con conjuntos de datos grandes o complejos. Pandas te ofrece dos herramientas para realizar esta selección con precisión: loc e iloc.

- 1.El método loc te permite seleccionar filas o columnas por su nombre o etiqueta. Es ideal cuando conoces los nombres específicos de las filas o columnas que deseas extraer. Su sintaxis es sencilla: `df.loc[filas, columnas]`.
- 2.El método iloc te permite seleccionar filas o columnas por su índice numérico. Es ideal cuando conoces la posición exacta de las filas o columnas que deseas extraer. Su sintaxis es sencilla: `df.iloc[filas, columnas]`.



Seleccionar columnas y filas por su nombre

En este caso, emplearemos el nombre de las filas o columnas que queremos extraer:

Extraer la segunda y tercera columna del Dataframe con todas las filas

```
data.loc[:,["nombre_columna2","nombre_columna3"]]
```

```
data.loc[:,["Element","Symbol"]]
```

Seleccionar columnas y filas por index

En el caso de querer usar el index, debes recordar que en Python empieza en 0, no en 1. Al igual que en el ejemplo anterior, si quieres seleccionar todas, usarás dos puntos ::

Extraer la fila 0 y todas las columnas

```
data.iloc[0,:]
```

Extraer todas las filas y las columnas de la 5 (incluida) a la 8 (excluida)

```
data.iloc[:,5:8]
```

Extraer las filas de la 2 (incluida) a la 6 (excluida) y todas las columnas

```
data.iloc[2:6,:]
```



Seleccionar columnas y filas mezclando index y nombres

También se pueden combinar ambos métodos, encadenando `.loc` e `.iloc` para hacer la selección de los datos.

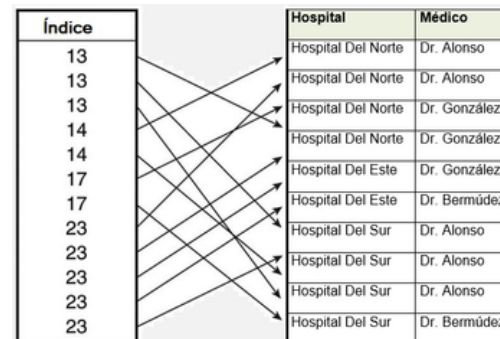
Extraer los datos de las filas 1 (incluida) a 3 (excluida) y de las columnas "Element" y "Symbol"

```
data.iloc[1:3, :].loc[:, ["Element", "Symbol"]]
```

```
data.loc[:, ["Element", "Symbol"]].iloc[1:3, :]
```

Extraer los datos de las filas 0 (incluida) a 3 (excluida) y las columnas 3 (incluida) a 6 (excluida)

```
data.iloc[0:3, :].iloc[:, 3:6]
```



Selecciones condicionales

En el análisis de datos con Pandas, la selección de datos no se limita a extraer filas o columnas específicas. Con las selecciones condicionales, puedes obtener solo los datos que cumplen con ciertos criterios, permitiéndote enfocarte en la información que realmente importa para tu análisis.

Filtrado por condiciones

Para realizar selecciones condicionales, se utiliza una expresión booleana que define la condición que deben cumplir los datos para ser incluidos. Esta expresión se aplica a una columna específica del DataFrame.

Extraer filas que tiene una masa atómica mayor que 280

```
data.loc[data.AtomicMass > 280,:]
```

```
data.loc[data.AtomicMass > 280,:].iloc[:,1:3]
```



Combinación de operadores booleanos

Puedes combinar operadores booleanos (&, |, ~) para crear expresiones más complejas y filtrar datos con mayor precisión.

#Extraer filas que tienen una masa atómica mayor que 280 y son de tipo Transactinide

```
data.loc[data.AtomicMass > 280]&(data.Type == "Transactinide"),:]
```

Examples for a dataset named sales:

`sales.loc[:, ["City", "Country", "Amount"]]`

Id	City	Country	Date	Amount
1	Madrid	SP	13-mar-20	124
2	Madrid	SP	15-mar-20	45
12	Seville	SP	13-mar-20	76
7	Madrid	SP	13-mar-20	35
23	Lisbon	PT	14-mar-20	146
19	Seville	SP	13-mar-20	541
134	Seville	CO	15-mar-20	19

`sales.iloc[:, [0,2,3]]`

Id	City	Country	Date	Amount
1	Madrid	SP	13-mar-20	124
2	Madrid	SP	15-mar-20	45
12	Seville	SP	13-mar-20	76
7	Madrid	SP	13-mar-20	35
23	Lisbon	PT	14-mar-20	146
19	Seville	SP	13-mar-20	541
134	Seville	CO	15-mar-20	19

`sales.iloc[2:5, :]`

Id	City	Country	Date	Amount
1	Madrid	SP	13-mar-20	124
2	Madrid	SP	15-mar-20	45
12	Seville	SP	13-mar-20	76
7	Madrid	SP	13-mar-20	35
23	Lisbon	PT	14-mar-20	146
19	Seville	SP	13-mar-20	541
134	Seville	CO	15-mar-20	19

`sales.iloc[1:4, :].loc[:, ["Id", "Amount"]]`

Id	City	Country	Date	Amount
1	Madrid	SP	13-mar-20	124
2	Madrid	SP	15-mar-20	45
12	Seville	SP	13-mar-20	76
7	Madrid	SP	13-mar-20	35
23	Lisbon	PT	14-mar-20	146
19	Seville	SP	13-mar-20	541
134	Seville	CO	15-mar-20	19

`sales.loc[sales.Amount>100, :]`

Id	City	Country	Date	Amount
1	Madrid	SP	13-mar-20	124
2	Madrid	SP	15-mar-20	45
12	Seville	SP	13-mar-20	76
7	Madrid	SP	13-mar-20	35
23	Lisbon	PT	14-mar-20	146
19	Seville	SP	13-mar-20	541
134	Seville	CO	15-mar-20	19

`sales.loc[sales.Amount>100, :].iloc[:, [0,1,4]]`

Id	City	Country	Date	Amount
1	Madrid	SP	13-mar-20	124
2	Madrid	SP	15-mar-20	45
12	Seville	SP	13-mar-20	76
7	Madrid	SP	13-mar-20	35
23	Lisbon	PT	14-mar-20	146
19	Seville	SP	13-mar-20	541
134	Seville	CO	15-mar-20	19

Manipulación

La manipulación de datos es una parte esencial del análisis, que va más allá de la simple selección de filas y columnas. Una tarea común y fundamental es renombrar columnas para que los nombres sean más descriptivos, fáciles de entender y adaptados a las necesidades de tu proyecto. Esto no solo mejora la claridad de los datos, sino que también facilita el proceso de análisis. Pandas ofrece herramientas simples y eficientes para realizar esta tarea, como la función `rename`.

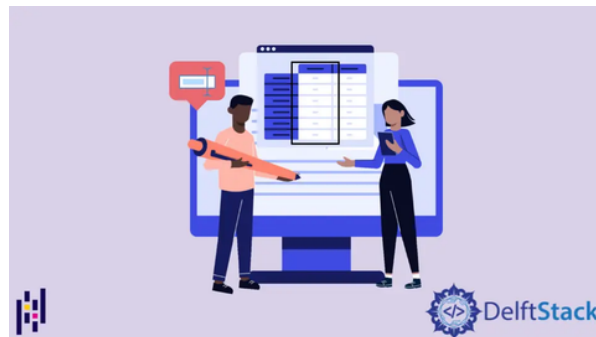


Renombrar una columna

Si necesitas cambiar el nombre de una única columna, puedes utilizar la función `rename` de Pandas. Esto se hace pasando un diccionario con el nombre actual de la columna como clave y el nuevo nombre como valor. También es importante usar el argumento `inplace=True` para que el cambio se aplique directamente al DataFrame sin necesidad de reasignarlo a una nueva variable.

Esta es la sintaxis básica:

```
df.rename(columns={"nombre_columna_actual": "nombre_columna_nuevo"}, inplace=True)
```



Renombrar más de una columna

Puedes renombrar múltiples columnas simultáneamente, usando un diccionario que incluya todos los cambios que deseas hacer.

Esta es la sintaxis básica:

```
df.rename(columns={"columna1": "nuevo_nombre1", "columna2": "nuevo_nombre2", ...}, inplace=True)
```

Por ejemplo, puedes renombrar la columna "ingresos" por "Income" y "Nombre" a "Name" de la siguiente manera:

```
clients.rename(columns={"ingresos": "Income", "Nombre": "Name"}, inplace=True)
```

Nombre	Apellido	Edad	ingresos
David	Smith	34	1200
Sara	Will	62	1400



Name	Apellido	Edad	Income
David	Smith	34	1200
Sara	Will	62	1400

Renombrar todas las columnas

Si necesitas renombrar todas las columnas del DataFrame, puedes hacerlo asignando una lista con los nuevos nombres a la propiedad `columns`. Es importante asegurarte de que la lista contenga exactamente el mismo número de nombres que el número de columnas en el DataFrame.

```
# Renombrar todas las columnas a "Name", "Surname", "Age" e "Income"  
clients.columns=["Name","Surname","Age","Income"]
```

Nombre	Apellido	Edad	ingresos
David	Smith	34	1200
Roberto	Kent	45	2100



Name	Surname	Age	Income
David	Smith	34	1200
Roberto	Kent	45	2100

Duplicados

Uno de los problemas más comunes al trabajar con conjuntos de datos es la presencia de registros duplicados. Estos duplicados pueden provenir de registros completamente repetidos o de elementos específicos que aparecen varias veces con diferentes identificadores. Independientemente de su origen, la eliminación de registros duplicados es esencial para asegurar que los resultados de tu análisis sean precisos y confiables.



Identificación de duplicados

Pandas ofrece varias herramientas para identificar y eliminar registros duplicados, lo que permite limpiar y preparar los datos para un análisis más preciso.

Pero antes de eliminar registros duplicados, es fundamental identificarlos. Puedes utilizar la función `uplicated()` para verificar si las filas en un DataFrame están duplicadas. Esta función devuelve una serie booleana donde `True` indica que una fila es un duplicado.

Identificar filas duplicadas

```
data.duplicated()
```

Una vez identificados los duplicados, puedes eliminarlos utilizando la función `drop_duplicates()`. Esta función elimina todas las filas duplicadas, manteniendo solo la primera aparición de cada duplicado.

Eliminar filas duplicadas

```
data.drop_duplicates(inplace=True)
```



	A
1	Name
2	Rachel
3	Rose
4	Susie
5	Susie
6	Tedi
7	Rachel
8	Rachel
9	Rose
10	Rose
11	Henrik
12	Wendy
13	Henrik
14	Henrik
15	Susie

	A
1	Name
2	Rachel
3	Rose
4	Susie
5	Susie
6	Tedi
7	Rachel
8	Rose
9	Rose
10	Rose
11	Henrik
12	Wendy
13	Henrik
14	Henrik
15	Susie

Mantener uno de los valores duplicados

Por defecto, el método `drop_duplicates()` mantiene la primera aparición de los datos duplicados, pero, en ocasiones, no es la que quiere mantener el programador. En este caso, emplearemos la propiedad **keep** de este método, que nos permite indicar si queremos conservar la primera o la última ocurrencia del duplicado. Si queremos especificar en qué columna debe buscar los duplicados Pandas, lo indicaremos con **subset=["nombre_columna"]**.

- **Valores repetidos**

Eliminar los duplicados en la columna Discoverer y mantener el primero

```
data.drop_duplicates(subset=["Discoverer"],keep="first")
```

- **Reducción de las ocurrencias.** En este caso, indicaremos a Pandas que elimine los duplicados únicamente cuando las columnas que indiquemos entre corchetes `[]` y separadas por comas, tengan los mismos valores en diferentes filas. Esta acción puede ser muy útil con nombres y fechas, donde es habitual mantener únicamente el último registro por considerarse el más actualizado.

Eliminar los duplicados en las columnas Discoverer y Year sólo si ambas columnas cuentan con un duplicado. Mantener la primera aparición

```
data.drop_duplicates(subset=["Discoverer","Year"],keep="first")
```

Eliminación de datos

La eliminación de datos no deseados es un proceso crucial para preparar tus datos para el análisis. Pandas ofrece la función `drop()` para eliminar filas o columnas de manera precisa y eficiente. Dependiendo de qué datos se quieran eliminar, hay diferentes formas de utilizar esta función.

- **Filas con valores faltantes (NaN):** Elimina filas que contienen valores ausentes o no válidos, asegurando que solo se analice información completa.
- **Columnas irrelevantes:** Elimina columnas que no aportan valor al análisis o que están duplicadas, simplificando el conjunto de datos y el proceso de análisis.
- **Registros erróneos:** Elimina registros que contengan errores o datos inconsistentes para garantizar la precisión de los resultados.

El elemento **axis** = especifica si se quieren eliminar filas (0) o columnas (1), mientras que **inplace=True** modifica el Dataframe original, eliminando los datos de forma permanente. Si quieres eliminar más de una columna, ponlas entre corchetes `[]` y separadas por comas `,`.



Eliminar una columna o más

Esta técnica permite eliminar una o varias columnas de un DataFrame. Puedes especificar las columnas por nombre o por índice.

Eliminar la columna 2 y la 4, habiendo declarado con anterioridad que forman parte de un mismo grupo al que se ha llamado "lista_de_variables"

```
lista_de_variables = ["Column2", "Column4"]
```

```
drop(lista_de_variables, axis=1, inplace=True)
```

En el ejemplo anterior, se crea una lista llamada lista_de_variables que contiene los nombres de las columnas que se quieren eliminar ("Column2" y "Column4"). Luego, se utiliza el método drop() para eliminar estas columnas del DataFrame data. El parámetro axis=1 indica que se eliminan columnas y inplace=True indica que la modificación se realiza en el DataFrame original.

Eliminar la columna "Element"

```
data.drop ("Element",axis=1 ,inplace=True)
```

En el ejemplo anterior se elimina una columna específica de un DataFrame por su nombre.

Eliminar las columnas "AtomicMass" y "Symbol"

```
data.drop (["AtomicMass","Symbol"], axis=1, inplace=True)
```



Eliminar todas las columnas que tengan valores faltantes (Na, NaN)

En este ejemplo se utiliza el método `dropna()` para eliminar todas las columnas del DataFrame `data` que contienen valores faltantes (NaN). El parámetro `axis=1` indica que se eliminan columnas.

```
data.dropna(axis=1)
```

Eliminar todas las filas donde haya valores faltantes (Na, NaN) en una columna

En este ejemplo se utiliza el método `dropna()` para eliminar todas las filas del DataFrame `data` que contienen valores faltantes (NaN) en la columna "Type". El parámetro `subset=["Type"]` indica que solo se deben considerar los valores faltantes en la columna "Type" para la eliminación de filas. El parámetro `inplace=True` indica que la modificación se realiza en el DataFrame original.

```
# Eliminar filas únicamente si el valor NaN está en la columna Type
```

```
data.dropna(subset=["Type"], inplace=True)
```



Valores NaN

En Pandas, los valores NaN (Not a Number) son valores especiales que representan datos faltantes o no computables. Estos valores se utilizan para denotar información que no se conoce, que no se puede calcular, o que ha sido eliminada.

Cuando se trabaja con conjuntos de datos en Pandas, es común encontrar valores NaN por diversas razones, como errores en la entrada de datos o la falta de disponibilidad de información durante la recopilación. Pandas maneja estos valores de manera específica, ofreciendo funciones y métodos para detectarlos, filtrarlos y tratarlos dentro de un conjunto de datos.

	myTableID	nombre	zona
1	4	Tarik	NULL
2	5	Herman	NULL
3	6	Allen	NULL
4	7	George	NULL
5	8	Ciaran	NULL
6	9	Eagan	NULL
7	10	Drake	NULL

Manejar valores nulos

Los DataFrames en Pandas cuentan con el método `isnull()`, que devuelve un DataFrame con las mismas dimensiones que el original, pero con valores booleanos que indican si los registros originales son nulos o no.

Para identificar si hay algún valor NaN en el DataFrame, puedes utilizar el siguiente código, que devolverá "True" si encuentra alguno:

```
data.isnull().values.any()
```

Si deseas ver los valores nulos en un DataFrame, puedes usar las siguientes líneas de código, que devolverán un DataFrame con "True" o "False" dependiendo de si el valor es nulo o no:

```
data.isnull()
```

```
data.notnull()
```

Para obtener el número de registros nulos por columna, puedes utilizar el método `sum()`. Si quieres aplicar esta operación a las filas, añade `axis=0`:

```
data.isnull().sum()
```

```
data.isnull().sum(axis=0)
```



Si necesitas obtener el número total de registros nulos en el DataFrame, utiliza esta línea de código:

```
data.isnull().sum().sum()
```

Puedes identificar si hay valores nulos en una variable específica, como se muestra a continuación:

```
data.Metal.isnull()
```

Para saber en qué filas hay valores NaN, utiliza este código, que devolverá "True" o "False" dependiendo de lo que encuentre:

```
data.isnull().any()
```

Finalmente, si quieres localizar las columnas que contienen valores NaN, utiliza el siguiente código, que devolverá "True" o "False" según lo que encuentre:

```
data.columns[data.isnull().any()]
```



Reemplazar valores NaN

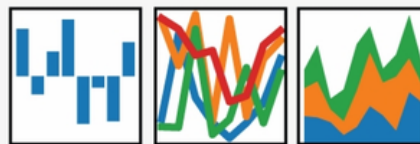
Si hay valores NaN en el DataFrame que impiden el análisis o interfieren con algún proceso, se pueden reemplazar por otros valores utilizando el método `fillna()`. A continuación, se detallan algunas formas de hacerlo:

- **Reemplazar todos los valores NaN del DataFrame**

Para cambiar cada valor NaN en todo el DataFrame por un valor específico, utiliza el siguiente código:

```
# Sustituir cada valor NaN por el valor 0  
data.fillna(value=0,inplace=True)
```

```
# Sustituir cada valor NaN por el valor 444444  
data.fillna(value=444444,inplace=True)
```



$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

pandas

Reemplazar los valores NaN en una columna específica

En este caso, queremos cambiar únicamente los valores que aparecen en la columna que especificamos. En **data.variable** indicamos en qué columna vamos a trabajar, mientras que en **value=value** especificamos qué valor vamos a emplear para sustituir los valores NaN:

```
# Sustituir valores NaN en una columna en concreto
```

```
data.variable.fillna(value=value,inplace=True)
```

```
# Sustituir cada valor NaN de la columna Year por el valor 2022
```

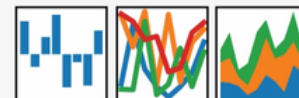
```
data.Year.fillna(value=2022,inplace=True)
```

Reemplazar los valores NaN de una variable específica

Para sustituir los valores NaN en una columna específica del DataFrame, usa:

```
# Sustituir los valores NaN de la variable Electronegativity con el valor 0
```

```
data["Electronegativity"].fillna(0, inplace = True)
```



$$y_i t = \beta' x_{it} + \mu_i + \epsilon_{it}$$

pandas

- **Reemplazar los valores NaN con el promedio de cada columna**

Para reemplazar los valores NaN en cada columna del DataFrame con el promedio de esa columna, utiliza:

```
# Sustituir los valores NaN de cada columna del DataFrame con el promedio de dicha columna  
data.fillna(value=data.mean(), inplace=True)
```

- **Reemplazar los valores NaN con una cadena específica**

Cuando los valores NaN representan datos faltantes debido a errores en la entrada, puedes reemplazarlos con una cadena como "No Info":

```
#Sustituir los valores NaN de la columna Metal por No info  
data["Metal"].fillna("No Info", inplace = True)
```

- **Reemplazar los valores NaN de una columna con el promedio de esa misma columna**

```
# Sustituir los valores NaN de la columna MeltingPoint con la media de los valores de la columna  
MeltingPoint  
data['MeltingPoint'].fillna(value=data['MeltingPoint'].mean(),inplace=True)
```



Extract y sort_values

Objetivo

Es comprender en la programación, los métodos `extract()` y `sort_values()` son muy útiles para seleccionar y organizar los datos necesarios para un análisis. En el caso de `extract` te permite buscar y extraer patrones de texto dentro de series de datos de tipo string. Por su parte, `sort_values` te permite ordenar los datos de un DataFrame en función de una o más columnas, permitiéndote organizar y visualizar la información de la manera que te resulte más útil.



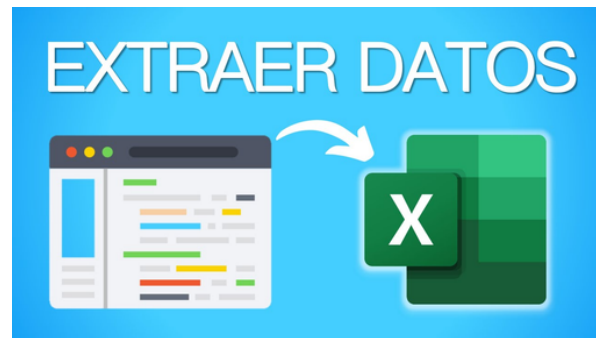
Extracción por tipo de variable (objeto, float, int...).

Este código crea un nuevo DataFrame `data_object` que contiene solo las columnas del DataFrame original que son de tipo objeto.

```
# Extraer las variables object  
data.select_dtypes(include=[object])
```

Este código crea un nuevo DataFrame `data_float` que contiene solo las columnas del DataFrame original que son de tipo float.

```
# Extraer las variables float  
data.select_dtypes(include=[float])
```



Extracción por rango numérico

Este código crea un nuevo DataFrame que contiene solo las filas del DataFrame original donde el valor de la columna "AtomicNumber" está entre 25 y 30 (ambos inclusive).

Extraer las filas donde la variable AtomicNumber esté entre el 25 y el 30 (ambas inclusive)

```
data[data["AtomicNumber"].between(25, 30)]
```

Extracción por palabra específica

Este código crea un nuevo DataFrame que contiene solo las filas del DataFrame original donde la palabra "gas" se encuentra en al menos una de las columnas "Phase", "Element" o "AtomicNumber".

Extraer las filas que contengan la palabra gas en las columnas Phase, Element y AtomicNumber

```
data.loc[:,["Phase","Element","AtomicNumber"]].loc[data.Phase.str.contains("gas"),:]
```



Extracción de la primera letra de cada palabra

Este código crea una nueva columna con la primera letra de las palabras. La letra que extrae se indica con `str[numero_de_letra]`.

```
#Extraer la primera letra de las palabras en la columna Discoverer  
data["letra"] = data["Discoverer"].astype(str).str[0]
```

Extracción de múltiples valores usando el método `isin`

Este código crea un nuevo DataFrame que contiene solo las filas del DataFrame original donde el valor de la columna "Phase" es "gas" o "solid".

```
# Extraer los valores gas y solid de las columnas Phase y Symbol  
data.loc[:,["Phase","Symbol"]].loc[data.Phase.isin(["gas","solid"]),:]
```

Extracción de la primera palabra de una columna

Este código crea una nueva columna con la palabra que ha identificado como la primera de cada columna. La palabra que se extrae se indica con `str.get(numero_de_palabra)`

```
# Extraer laprimera palabra de la columna Discoverer  
data["NEW"] = data.Discoverer.str.split().str.get(0)
```



Ejemplos de sort_values

Esta función ordena el Dataframe en orden ascendente (ascending = 1) o descendente (ascending = 0) por los valores de la columna especificada a lo largo de cualquiera de los índices. Se puede aplicar a una única variable o a más de una.

Ascender por una variable de menor a mayor

Ordenar el Dataframe de forma ascendente en base a los valores de la columna Year. Añadir las columnas Element y AtomicNumber para que aparezcan en el Dataframe que se va a crear.

`data.loc[:,["Year","Element","AtomicNumber"]].sort_values(by="Year",ascending =1)`

Ascendiendo por más de una variable.

Ordenar el Dataframe de forma ascendente en base a los valores de la columna Year. Dentro de ese orden, organizar en base al número de isótopos. Añadir la columna Elements para que aparezca en el Dataframe que se va a crear.

`data.loc[:,["Year","Element","NumberOfIsotopes"]].sort_values(by=["Year",
,"NumberOfIsotopes"],ascending =[1,0])`

Examples for a dataset named rests:

rests	id	City	Country	Income	Employees
	1	Madrid	SP	1000	10
	2	London	UK	1200	2
	3	Lisbon	PT	900	5
	4	Madrid	SP	600	8
	5	Seville	SP	600	5
	6	Lisbon	PT	1200	7

rests.sort_values(by = ["Income"], ascending = 0)	id	City	Country	Income	Employees
	2	London	UK	1200	2
	6	Lisbon	PT	1200	7
	1	Madrid	SP	1000	10
	3	Lisbon	PT	900	5
	4	Madrid	SP	600	8
	5	Seville	SP	600	5

rests.sort_values(by = ["Income","Employees"], ascending = [0,1])	id	City	Country	Income	Employees
	2	London	UK	1200	2
	6	Lisbon	PT	1200	7
	1	Madrid	SP	1000	10
	3	Lisbon	PT	900	5
	5	Seville	SP	600	5
	4	Madrid	SP	600	8

Análisis estadístico básico

Cuando se trabaja con números, es muy probable que la estadística entre en juego en algún momento. Pandas ofrece ciertas funcionalidades para trabajar con elementos estadísticos mediante el uso de comandos. Algunos de los más usados son:

- **df.max():** Calcula el valor máximo.
- **df.min():** Calcula el valor mínimo.
- **df.mean():** Calcula la media de los valores que se indiquen.
- **df.sum():** Suma todos los valores que se indiquen.
- **df.variable.value_counts():** Calcula la frecuencia de aparición de cada valor único indicado.
- **df.count():** Realiza un conteo de todos los valores no nulos en cada columna.



Correlaciones

La correlación es una medida estadística que cuantifica la fuerza y la dirección de la relación lineal entre dos variables. Esto no implica necesariamente que existe una relación causal entre las variables. Existen dos tipos de correlación:

- **Correlación positiva:** Indica que dos variables se mueven en la misma dirección. Un ejemplo básico es la relación entre peso y altura, donde las personas más altas tienden a pesar más y viceversa.
- **Correlación negativa:** Indica que dos variables se mueven en direcciones opuestas. Un ejemplo básico es la relación entre velocidad y distancia, donde a mayor velocidad, menor tiempo se necesitará para llegar al destino y viceversa.

Matriz de correlación

Sepal.Length	1	-0.12	0.87	0.82
Sepal.Width	-0.12	1	-0.43	-0.37
Petal.Length	0.87	-0.43	1	0.96
Petal.Width	0.82	-0.37	0.96	1

Ejemplos de correlación

- **Correlación entre dos variables**

Averiguar la correlación entre las variables AtomicMass y AtomicRadius

```
data.AtomicMass.corr(data.AtomicRadius)
```

- **Correlación de una variable con las demás**

Averiguar la correlación de la variable AtomicMass con el resto

```
#data.corrwith(data.AtomicMass)
```

- **Correlación en todo el dataset**

Averiguar la correlación de las variables de todo el dataset

```
data.corr()
```



Group_by

Otra de las maneras de organizar los datos en Pandas es el método `group_by`, que permite agrupar datos en un Dataframe según los valores de una o varias columnas. Este es un concepto clave, ya que cuenta con la capacidad de resumir, agregar y agrupar datos de manera eficiente.

Este método permite crear una agrupación de categorías y aplicarles una función, que es una técnica ampliamente usada en la ciencia de datos. Un proyecto dentro de este ámbito tratará con grandes cantidades de datos y necesitará probar cosas una y otra vez, lo que puede realizarse de forma más eficiente mediante `group_by`.



Agrupar por una variable y sacar la suma de todas las demás variables numéricas. Las variables numéricas que se sumarán son las creadas tras la agrupación.

#Agrupar por la variable Phase y extraer la suma de las variables numéricas que surgen

```
data.groupby(by=["Phase"],as_index=False).sum()
```

Agrupar por dos variables y sacar la media de todas las demás variables numéricas

Agrupar por las variables Phase y AtomicMass y extraer la media de las variables numéricas que surgen

```
data.loc[:,["Phase","AtomicMass"]].groupby(by=["Phase"],as_index=False).mean()
```

Agrupar por tres variables y contar cuántas veces aparece cada valor en una variable

Agrupar por las variables Discoverer, Phase y Type y contar cuántas veces aparece cada valor en una variable

```
data.loc[:,["Discoverer","Phase","Type"]].groupby(by= ["Discoverer","Type"],as_index=False).count()
```



¡Sigamos trabajando!

